

Lesson 3: Decisions

Fundamental Programming Concepts in Python

Faculty of Data Science and Artificial Intelligence

National Economics University

DSAI1003

Based on Chapter 3 – *Decisions, Python for Everyone.*

Why Do We Need Decisions?

In Lesson 2, programs mostly executed statements from top to bottom.

Real programs need to **make decisions**: choose different actions depending on data.

Examples

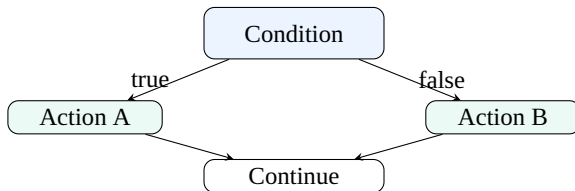
- apply a discount;
- check user input;
- classify a grade;
- choose a tax formula;
- process strings conditionally.

The Central Idea

Evaluate a condition $\rightarrow$ **True/False** $\rightarrow$ **Choose a branch**

Control flow

Condition is true $\Rightarrow$ execute action A
Condition is false $\Rightarrow$ execute action B



Learning Objectives

After this lesson, students should be able to:

- use `if`, `if/else`, and `if/elif/else`;
- compare values with relational operators;
- combine conditions with `and`, `or`, and `not`;
- design nested branches and multiple alternatives;
- hand-trace programs with branches;
- design test cases for branches and boundary values;
- analyze strings and validate input.

Lesson Structure

- 1 The `if` statement
- 2 Relational operators
- 3 Nested branches
- 4 Multiple alternatives
- 5 Flowcharts and test cases
- 6 Boolean variables and operators
- 7 Analyzing strings
- 8 Input validation
- 9 Summary, quizzes, and practice exercises

A Simple if Statement

An `if` statement executes a block of code only when a condition is true.

```
if score >= 50 :  
    print("Pass")
```

Key points

- The condition follows the keyword `if`.
- The line ends with a colon `:`.
- The controlled statement block is indented.

Syntax Pattern

```
if condition :  
    statement1  
    statement2  
    ...
```

Reading the structure

- If the condition is true, execute the indented block.
- If the condition is false, skip the indented block.
- Execution continues after the block.

Example: Elevator Floor Number

Some buildings skip floor 13. If a user enters a floor above 13, the actual floor is one less.

```
floor = int(input("Floor: "))

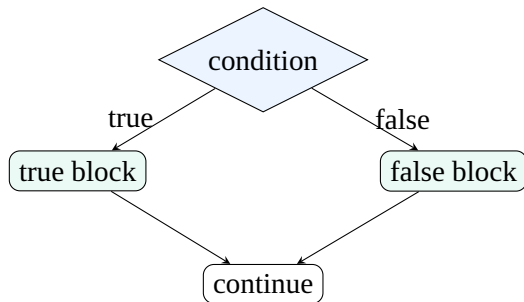
if floor > 13 :
    actualFloor = floor - 1
else :
    actualFloor = floor

print("The elevator will travel to floor", actualFloor)
```

Two-Way Decision

if/else

Use **else** when there are two mutually exclusive actions.



Example: Discount

```
price = float(input("Price: "))

if price >= 100 :
    discount = 0.10 * price
else :
    discount = 0.00

finalPrice = price - discount
print("Final price: %.2f" % finalPrice)
```

Observation

Only one of the two blocks runs.

Common Error: Tabs and Spaces

Python uses indentation to define blocks. Inconsistent indentation changes program meaning or causes an error.

Avoid

Do not mix tabs and spaces. Use spaces consistently.

Good habit

Use 4 spaces per indentation level.

Programming Tip: Avoid Duplication

Poor style:

```
if price >= 100 :  
    discount = 0.10 * price  
    finalPrice = price - discount  
    print(finalPrice)  
else :  
    discount = 0.00  
    finalPrice = price - discount  
    print(finalPrice)
```

Better: put common statements after the branch.

Avoid Duplication: Improved Version

```
if price >= 100 :  
    discount = 0.10 * price  
else :  
    discount = 0.00  
  
finalPrice = price - discount  
print(finalPrice)
```

Reason

Duplicated code is harder to maintain and easier to update inconsistently.

Quiz 3.1: if/else

What does this program print when $x = 7$?

```
x = 7
if x > 10 :
    print("Large")
else :
    print("Small")
print("Done")
```

- ① Large / Done
- ② Small / Done
- ③ Done only
- ④ Error

Answer to Quiz 3.1

Correct answer: 2

The condition `x > 10` is false when `x = 7`, so Python executes the `else` block.

Small then Done

Relational Operators

Operator	Meaning
==	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Important

Use == for equality testing. Use = for assignment.

Assignment vs Equality Test

Assignment

```
x = 5
```

Stores the value 5 in x.

Equality test

```
x == 5
```

Checks whether x is equal to 5.

Example: Comparing Numbers

```
balance = float(input("Balance: "))
minimum = 1000.0

if balance < minimum :
    print("Below minimum balance")
else :
    print("Balance is acceptable")
```

Condition result

A relational expression produces a Boolean value: True or False.

Comparing Strings

Strings can also be compared.

```
word = input("Word: ")  
if word == "Python" :  
    print("Correct")  
else :  
    print("Different")
```

Case sensitivity

"Python" and "python" are different strings.

Lexicographic Ordering

Strings are ordered lexicographically, similar to dictionary order.

Expression	Result
"Harry" < "Harris"	False
"apple" < "banana"	True
"Zoo" < "apple"	depends on character codes

Use with care

Uppercase and lowercase letters have different character codes.

Floating-Point Comparison

Floating-point calculations may contain tiny roundoff errors.

```
from math import sqrt
r = sqrt(2.0)
print(r * r)
```

The result may be close to, but not exactly, `2.0`.

Common error

Do not rely on exact equality for computed floating-point values when approximation is expected.

Testing “Close Enough”

```
EPSILON = 1E-14

if abs(x - y) <= EPSILON :
    print("Close enough")
else :
    print("Different")
```

Idea

Compare the magnitude of the difference with a small tolerance.

How To: Implement an `if` Statement

A systematic workflow:

- ➊ Decide what condition must be tested.
- ➋ Express the condition using relational operators.
- ➌ Decide what to do when the condition is true.
- ➍ Decide what to do when the condition is false.
- ➎ Translate the structure into Python.

Worked Example: Extracting the Middle

Problem: Given three numbers, print the value that is neither the smallest nor the largest.

Sample

For 7, 2, 5, the middle value is 5.

Strategy

Check which number lies between the other two.

Extracting the Middle: Code

```
x = int(input("First number: "))
y = int(input("Second number: "))
z = int(input("Third number: "))

if (x <= y and y <= z) or (z <= y and y <= x) :
    middle = y
elif (y <= x and x <= z) or (z <= x and x <= y) :
    middle = x
else :
    middle = z

print("The middle value is", middle)
```

Example: Shipping Fee

```
amount = float(input("Order amount: "))

if amount >= 50.0 :
    shipping = 0.0
else :
    shipping = 5.95

total = amount + shipping
print("Shipping: %.2f" % shipping)
print("Total: %.2f" % total)
```

Quiz 3.2: Relational Operators

Which expression tests whether x is between 0 and 100, including both endpoints?

- ❶ $x > 0$ and $x < 100$
- ❷ $x \geq 0$ and $x \leq 100$
- ❸ $x \geq 0$ or $x \leq 100$
- ❹ $0 \leq x \leq 100$ only works in mathematics, not Python

Answer to Quiz 3.2

Correct answers: 2 and also 4 is valid in Python

Python allows chained comparisons such as `0 <= x <= 100`. The longer form `x >= 0 and x <= 100` is also correct.

Common mistake

`x >= 0 or x <= 100` is almost always true and does not restrict `x` to the interval.

When Do We Need Nested Branches?

Use nested branches when one decision depends on the result of another decision.

Examples

- tax calculation: marital status first, then income;
- grade classification with special cases;
- input validation before computation;
- choosing one formula inside another category.

Nested `if`: Basic Pattern

```
if condition1 :  
    if condition2 :  
        statementA  
    else :  
        statementB  
else :  
    statementC
```

Reading the structure

The inner decision is considered only if the outer branch is entered.

Example: Tax with Two Groups

```
income = float(input("Income: "))
status = input("Single or married? ")

if status == "single" :
    if income <= 32000 :
        tax = 0.10 * income
    else :
        tax = 3200 + 0.25 * (income - 32000)
else :
    if income <= 64000 :
        tax = 0.10 * income
    else :
        tax = 6400 + 0.25 * (income - 64000)
```

Hand-Tracing

Hand-tracing means simulating a program step by step, usually on paper.

How to trace

- Make a column for each variable.
- Execute statements one at a time.
- Update variable values when they change.
- Record which branch is taken.

Why it helps

It reveals whether the program logic matches the intended logic.

Tracing Example

Input: `income = 80000, status = "married"`

Step	Branch	Limit	Tax
1	married branch	64000	—
2	income above limit	64000	—
3	compute tax	64000	$6400 + 0.25 * (16000)$
4	result	—	10400

Example: Login Check

```
username = input("Username: ")
password = input("Password: ")

if username == "admin" :
    if password == "python" :
        print("Welcome")
    else :
        print("Wrong password")
else :
    print("Unknown user")
```

Discussion

The password is checked only for the recognized user.

Quiz 3.3: Nested Branches

A program first checks whether a purchase amount is positive. Only if it is positive, it checks whether a discount applies. Which structure is most natural?

- ❶ two independent `if` statements
- ❷ an `if` statement inside another `if`
- ❸ only an `else` block
- ❹ no branch is needed

Answer to Quiz 3.3

Correct answer: 2

The discount check should happen only after the amount is known to be valid and positive.

Multiple Alternatives with `elif`

Use `elif` when several alternatives are mutually exclusive.

```
if condition1 :  
    statement1  
elif condition2 :  
    statement2  
elif condition3 :  
    statement3  
else :  
    statement4
```

Only the first true branch is executed.

Example: Grade Classification

```
score = float(input("Score: "))

if score >= 90 :
    grade = "A"
elif score >= 80 :
    grade = "B"
elif score >= 70 :
    grade = "C"
elif score >= 60 :
    grade = "D"
else :
    grade = "F"

print("Grade:", grade)
```

Ordering Conditions

The order of `elif` conditions matters.

Wrong order

If `score >= 60` is tested first, then a score of 95 is classified as D.

Good order

Start with the most restrictive/highest threshold, then move downward.

Example: Earthquake Magnitude

```
magnitude = float(input("Magnitude: "))

if magnitude >= 8.0 :
    description = "Most structures fall"
elif magnitude >= 7.0 :
    description = "Many buildings destroyed"
elif magnitude >= 6.0 :
    description = "Many buildings damaged"
elif magnitude >= 4.5 :
    description = "Damage to poorly constructed buildings"
else :
    description = "No destruction of buildings"

print(description)
```

Example: Shipping by Region

```
region = input("Region: ").lower()
weight = float(input("Weight: "))

if region == "local" :
    rate = 1.5
elif region == "domestic" :
    rate = 3.0
elif region == "international" :
    rate = 8.0
else :
    rate = -1

if rate >= 0 :
    print("Fee: %.2f" % (rate * weight))
else :
    print("Unknown region")
```

Quiz 3.4: Choosing a Structure

Which structure is best for classifying a score into A/B/C/D/F?

- ❶ five unrelated `if` statements
- ❷ one `if/elif/else` chain
- ❸ only `else`
- ❹ a loop

Answer to Quiz 3.4

Correct answer: 2

The alternatives are mutually exclusive: one score should receive one grade.

Flowcharts

A flowchart is a visual representation of program control flow.

Shape	Meaning
Oval	start / end
Rectangle	action / computation
Diamond	decision
Arrow	direction of flow

Avoid Spaghetti Control Flow

A clear program has structured control flow.

Spaghetti control flow

A tangled flow of jumps and branches is difficult to understand, test, and maintain.

Better

Use structured `if`, `if/else`, `elif`, and later loops/functions.

Test Cases

A test case is a concrete input with an expected output.

For decision programs, test

- every branch;
- important boundary values;
- typical valid cases;
- invalid or unusual inputs.

Boundary Values

Boundary values are values near the point where behavior changes.

Grade example

Important values include 59, 60, 69, 70, 79, 80, 89, 90.

Reason

Many errors occur at the boundary between two branches.

Example: Test Table for Grade Classification

Input score	Expected grade	Purpose
95	A	typical A
90	A	boundary
89	B	just below A
80	B	boundary
70	C	boundary
60	D	boundary
59	F	just below passing

Quick Activity: Design Test Cases

Design test cases for this rule:

Rule

A customer gets free shipping if the order amount is at least \$50. Otherwise, shipping costs \$5.95.

Questions

- Which values are typical?
- Which values are boundary values?
- What invalid inputs might occur?

Suggested Test Cases

Amount	Expected shipping	Purpose
25.00	5.95	ordinary below threshold
49.99	5.95	just below threshold
50.00	0.00	boundary
75.00	0.00	ordinary above threshold
0.00	depends on policy	special case
-10.00	invalid	input validation

Boolean Values

Python has two Boolean values: True and False.

```
passed = score >= 60

if passed :
    print("Pass")
else :
    print("Fail")
```

Boolean variable

A Boolean variable stores the result of a condition.

The and Operator

and is true only when both conditions are true.

```
if age >= 18 and registered :  
    print("Can participate")  
else :  
    print("Cannot participate")
```

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

The or Operator

or is true when at least one condition is true.

```
if role == "admin" or role == "teacher" :  
    print("Access granted")  
else :  
    print("Access denied")
```

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

The not Operator

not reverses a Boolean value.

```
if not valid :  
    print("Invalid input")
```

A	not A
False	True
True	False

Operator Precedence

Boolean operators have precedence rules.

Priority	Operator
1	relational operators, e.g. <, ==
2	not
3	and
4	or

Tip

Use parentheses when the intended logic is not immediately obvious.

Short-Circuit Evaluation

Python stops evaluating a Boolean expression as soon as the result is known.

```
if quantity > 0 and price / quantity < 10 :  
    print("Good price")
```

Why this is useful

If `quantity > 0` is false, Python does not evaluate `price / quantity`, avoiding division by zero.

De Morgan's Laws

Useful for simplifying negated Boolean expressions.

Rules

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

Python form

`not (A and B)` is equivalent to `not A or not B`.

Example: Range Check

```
if value < 0 or value > 100 :  
    print("Outside range")  
else :  
    print("Inside range")
```

Equivalent idea:

```
if not (0 <= value <= 100) :  
    print("Outside range")
```

Quiz 3.5: Boolean Logic

For $x = 5$, which expression is true?

- ❶ $x < 0$ or $x > 10$
- ❷ $x \geq 0$ and $x \leq 10$
- ❸ $\text{not } (x == 5)$
- ❹ $x < 0$ and $x > 10$

Answer to Quiz 3.5

Correct answer: 2

$x = 5$ lies between 0 and 10, including both endpoints.

Why not the others?

The other expressions describe values outside the range, not equal to 5, or an impossible condition.

Membership Tests

Use `in` to test whether a substring appears in a string.

```
email = input("Email: ")  
  
if "@" in email :  
    print("Contains @")  
else :  
    print("Missing @")
```

Result

The expression `"@" in email` produces `True` or `False`.

String Methods for Analysis

Method	Meaning
<code>s.isdigit()</code>	all characters are digits
<code>s.isalpha()</code>	all characters are letters
<code>s.isupper()</code>	uppercase letters
<code>s.islower()</code>	lowercase letters
<code>s.startswith(t)</code>	starts with substring t
<code>s.endswith(t)</code>	ends with substring t

Normalize Case Before Comparing

```
answer = input("Continue? Y/N: ")
answer = answer.upper()

if answer == "Y" :
    print("Continue")
elif answer == "N" :
    print("Stop")
else :
    print("Invalid answer")
```

Reason

The program accepts y, Y, n, and N consistently.

Example: User Name Check

A simple user name rule: at least 5 characters and only letters/digits.

```
username = input("Username: ")

if len(username) < 5 :
    print("Too short")
elif not username.isalnum() :
    print("Use only letters and digits")
else :
    print("Valid username")
```

Quiz 3.6: String Analysis

What does this program print when the input is Python3?

- ① Too short
- ② Use only letters and digits
- ③ Valid username
- ④ Error

Given rule

Minimum length is 5 and `isalnum()` must be true.

Answer to Quiz 3.6

Correct answer: 3

Python3 has length 7 and contains only letters and digits, so it is valid.

Why Validate Input?

Programs should not assume that all user input is valid.

Possible problems

- missing required characters;
- number outside valid range;
- nonnumeric text where a number is expected;
- invalid menu choice;
- empty string.

Example: Validate a Percentage

```
rate = float(input("Enter interest rate: "))  
  
if rate < 0 or rate > 100 :  
    print("Invalid rate")  
else :  
    print("Rate accepted")
```

Boundary values

Test 0, 100, -1, and 101.

Example: Validate a Floor Number

```
floor = int(input("Floor: "))

if floor < 0 :
    print("Floor cannot be negative")
elif floor == 13 :
    print("There is no floor 13")
else :
    print("Floor accepted")
```

Intersecting Circles: Testing Idea

Two circles intersect if the distance between centers is not too large and not too small.

Given

Center distance d ; radii r_1 , r_2 .

Intersection condition

$$|r_1 - r_2| \leq d \leq r_1 + r_2$$

Intersecting Circles: Code

```
from math import sqrt

dx = x1 - x2
dy = y1 - y2
distance = sqrt(dx ** 2 + dy ** 2)

if abs(r1 - r2) <= distance and distance <= r1 + r2 :
    print("The circles intersect")
else :
    print("The circles do not intersect")
```

Quiz 3.7: Input Validation

Which input should be rejected for a percentage value?

- 1 0
- 2 50
- 3 100
- 4 120

Answer to Quiz 3.7

Correct answer: 4

A percentage value should normally be between 0 and 100, inclusive.

Summary: Decisions

- `if` executes a block only when a condition is true.
- `else` handles the false case.
- `elif` handles multiple mutually exclusive cases.
- Nested branches support multi-level decisions.
- Indentation defines the block structure.

Summary: Conditions and Boolean Logic

- Relational operators produce `True` or `False`.
- `and`, `or`, and `not` combine conditions.
- Short-circuit evaluation can prevent unnecessary or unsafe evaluations.
- De Morgan's laws help rewrite negated conditions.
- String methods are useful for validation and classification.

Summary: Testing and Validation

- Test every branch.
- Include boundary values.
- Use hand-tracing to understand control flow.
- Validate user input before processing it.
- Normalize text input when case should not matter.

Quick Quiz

- ❶ What is the difference between `=` and `==`?
- ❷ Why is indentation important in Python?
- ❸ When should `elif` be used?
- ❹ Why are boundary values important in testing?
- ❺ What does short-circuit evaluation mean?

In-Class Exercise 1: Classify a Number

Write a program that reads an integer and prints one of the following:

- negative
- zero
- positive

Suggested structure

Use an `if/elif/else` chain.

In-Class Exercise 2: Username Validation

Write a program that reads a username and checks:

- length is at least 5;
- it contains only letters and digits;
- it does not start with a digit.

Print a specific message for each invalid case.

Additional Comprehensive Quiz

Choose the best answer.

- 1 Which operator tests equality? `=`, `==`, or `!=`?
- 2 Which structure is best for A/B/C/D/F grading?
- 3 What is the opposite of `x < 0` or `x > 100`?
- 4 Why should we test 59, 60, 89, and 90 in a grading program?

Answers to the Comprehensive Quiz

- ① `==` tests equality.
- ② An `if/elif/else` chain.
- ③ `x >= 0` and `x <= 100`, or `0 <= x <= 100`.
- ④ They are boundary values where behavior changes.

Comprehensive Example: Order Classification

```
amount = float(input("Order amount: "))
member = input("Member? Y/N: ").upper()

if amount < 0 :
    print("Invalid amount")
elif member == "Y" and amount >= 100 :
    print("VIP discount")
elif member == "Y" :
    print("Member discount")
elif amount >= 100 :
    print("Large order")
else :
    print("Regular order")
```

Study Guidance

Practice cycle

Predict → Trace → Run → Explain → Modify

- First predict which branch will run.
- Then trace variables by hand.
- Run the program and compare with your prediction.
- Modify conditions and observe the effect.

End of Lesson 3

Questions?

Next lesson: Loops